

UNIT V

DISCOURSE ANALYSIS AND LEXICAL RESOURCES: Discourse segmentation, Coherence – Reference Phenomena, Anaphora Resolution using Hobbs and Centering Algorithm – Coreference Resolution – Resources: Porter Stemmer, Lemmatizer, Penn Treebank, Brill's Tagger, WordNet, PropBank, FrameNet, Brown Corpus, British National Corpus (BNC).

Discourse segmentation:

Discourse segmentation in NLP is the process of dividing continuous text or dialogue into coherent, smaller units (elementary discourse units or EDUs) based on topic shifts or logical structure. It enhances machine understanding for applications like text summarization, question answering, and dialogue systems by analyzing cues such as discourse markers.

Key Aspects of Discourse Segmentation

- **Definition:** It involves identifying the boundaries between different discourse units, such as sentences or paragraphs, to understand the structure of a larger text.
- **Purpose:** To break down complex, lengthy text into manageable, meaningful segments, which is essential for tasks like information retrieval and text summarization.

Methods:

- **TextTiling Algorithm:** A commonly used method for linear segmentation, which detects subtopic changes based on lexical cohesion.
- **Machine Learning Approaches:** These utilize various features such as syntactic cues and lexical patterns to determine segment boundaries.
- **Discourse Markers:** The identification of specific words or phrases that signal changes in topic or perspective.
- **Applications:**
 - **Text Summarization:** Creating shorter, more coherent versions of documents.
 - **Information Extraction:** Identifying key information in structured or unstructured data.
 - **Dialogue Systems:** Analyzing conversational flow and turn-taking in spoken dialogue.
 - **Machine Translation:** Improving accuracy by understanding the context and structure of the text.
- **Core Concepts:**
 - **Cohesion and Coherence:** Evaluating how well the text maintains a logical, ordered flow.
 - **Coreference Resolution:** Identifying expressions that refer to the same entity in different segments.

Discourse segmentation is a critical component of natural language processing that improves the ability of machines to comprehend and analyze the structure and meaning of text.

Coherence – Reference Phenomena

Coherence in NLP refers to the semantic, logical, and structural connectivity that makes a discourse meaningful, rather than just a sequence of sentences. It relies on entity-based relationships and discourse relations (e.g., cause, elaboration) to create a structured flow. Reference phenomena, such as anaphora, cataphora, and coreference resolution, are essential to maintaining this coherence by identifying when different expressions refer to the same entity (e.g., "Elon Musk" and "He").

Key Aspects of Coherence and Reference in NLP

- **Definition & Importance:** Coherence is a semantic property of discourse, determining how individual sentences are interpreted relative to others. It is essential for generating natural-sounding text, evaluating NLP output quality, and understanding discourse flow.
- **Reference Phenomena (Coreference Resolution):** This is the process of identifying when different words or phrases (mentions) refer to the same entity (e.g., "she" referring to "Ana"). It is crucial for maintaining coherence in dialogue systems, question-answering systems, and machine translation.
- **Coherence Relations:** These are the logical connections between utterances, such as:
 - **Cause/Effect:** The carpenter was tired. He had spent the day building a cabinet.
 - **Elaboration:** A statement providing more detail about a previous one.
 - **Contrast/Parallel:** Comparing or contrasting ideas.
- **Modelling Coherence:**
 - **Centering Theory:** A framework focusing on local coherence by tracking how entities (discourse centers) are introduced and maintained across sentences.
 - **Entity-Grid Approach:** A method to measure local coherence by tracking how entities appear across sentences.
- **Applications:**
 - **Automatic Essay Scoring:** Assessing the logical structure and flow of student essays.
 - **Dialogue Systems:** Ensuring the chatbot understands references to previous turns.
 - **Machine Translation:** Properly translating pronouns based on the context of the entire document.

Coherence vs. Cohesion

While cohesion links elements through grammatical features (like pronouns or conjunctions), coherence ensures the overall logical unity and meaning of the discourse.

Common Challenges

- **Ambiguity:** Determining which entity a pronoun refers to when multiple options are present.
- **Long-Distance Dependencies:** Resolving references that appear far apart in a text.
- **Contextual Understanding:** Inferring the underlying connection between sentences, which often requires world knowledge.

Anaphora Resolution using Hobbs and Centering Algorithm

Anaphora resolution in Natural Language Processing involves identifying the antecedent of pronouns or other referring expressions. The Hobbs Algorithm is a syntactic, rule – based method, while the Centering Algorithm is a discourse – based method that models attention shifts to resolve these references. Anaphora resolution in NLP is the process of identifying the specific antecedent (noun phrase) that a pronoun or referring expression (anaphor) points back to in a text. This essential discourse analysis technique enables machines to understand context, enhancing accuracy in machine translation, text summarization, and question-answering systems.

Key Concepts and Types

- **Anaphor:** The word or phrase (Ex:- “she” , “it”) that refers to a previous item.
- **Antecedent:** The initial entity (Ex: “Maruti”, “The car”) that the anaphor refers to.

Types of Anaphora:

- **Pronominal:** Using pronouns (Ex: “Rama bought a book. He read it.”).
- **Definite Noun Phrase:** Using a noun phrase (Ex: “I bought a car. The vehicle is fast”.)
- **Event:** Referring to a whole clause or event.
- **Cataphora:** When the referent appears after the pronoun (Ex: “Because he was tired, Rama slept”.)

Algorithm

The Hobbs algorithm (1978) is a rule-based, "knowledge-poor" approach that uses the syntactic parse tree of the sentences to find potential antecedents.

- **Approach:** It performs a left-to-right, breadth-first search up the parse tree from the anaphor (pronoun) node.
- **Process:**
 1. Start at the Noun Phrase (NP) node immediately dominating the pronoun.
 2. Go up the tree to the first NP or Sentence (S) node found.
 3. Conduct a breadth-first, left-to-right search of the children of that node to the left of the path taken up the tree, proposing any NP found as a potential antecedent.
 4. If the search fails, continue up the tree to the next S or NP node and repeat the search process, including searching previous sentences in order of recency.
- **Constraints:** It applies syntactic and simple semantic constraints, such as gender, number, and person agreement, to filter out unlikely candidates. It also uses the "Hobbs distance" property, which prioritizes entities in subject positions as more likely antecedents.
- **Advantage:** It is simple and relatively easy to implement.

Centering Algorithm

The Centering algorithm (based on Centering Theory) is a discourse-based method that focuses on how the speaker/writer shifts attention between entities in a conversation or text.

- **Approach:** It maintains a discourse model and assigns a degree of salience (importance) to potential referents based on factors like grammatical role and recency.
- **Process:** It defines "centers" (entities in focus) for each utterance (sentence), specifically the backward-looking center (Cb) and forward-looking centers (Cfs). It uses rules and constraints to determine how the centers shift between consecutive utterances.
- **Constraints:** It prioritizes the most salient referent (e.g., the subject of the sentence has higher salience) that matches the anaphor's features (gender, number, etc.).

Advantage: It models discourse coherence more effectively than purely syntactic methods, leading to potentially higher accuracy in complex cases.

Key Differences

S.No.	Features	Hobbs Algorithm	Centering Algorithm
1.	Approach	Primarily syntactic , using parse trees and search order.	Discourse-based , modeling attentional state and salience.
2.	Focus	Locality and grammatical rules within and across sentences.	Coherence and smooth shifts of attention across discourse segments.
3.	Mechanism	Breadth-first search for NP candidates.	Ranking of potential antecedents based on salience values and grammatical roles.
4.	Performance	A simpler, baseline algorithm.	Generally shows higher accuracy on corpora compared to the simpler Hobbs algorithm.

Coreference Resolution

Coreference Resolution (CR) is a crucial Natural Language Processing (NLP) task that identifies all expressions (mentions) in a text—such as pronouns ("she"), noun phrases ("the company"), or proper nouns—that refer to the same real-world entity. It is essential for understanding context and ambiguity in applications like machine translation, summarization, and information extraction.

Key Aspects of Coreference Resolution:

- **Goal:** To group linguistic expressions (mentions) that refer to the same entity, creating a chain of references.
- **Examples:** In "Sita went to the park. She enjoyed the fresh air," CR links "She" to "Sita".

Key Techniques:

- **Rule-based:** Uses linguistic rules (e.g., gender/number agreement).
- **Machine Learning/Deep Learning:** Models learn patterns from annotated data (e.g., Mention-Pair, Entity-Mention models).

Applications:

- **Information Extraction:** Connecting disparate facts about the same entity.
- **Text Summarization:** Ensuring summaries accurately reflect entity actions.
- **Dialogue Systems:** Tracking participants and objects in conversation.
- **Challenges:** Resolving long-distance dependencies, ambiguity, and interpreting diverse phrasing.

Common Coreference Types:

- **Pronoun Resolution:** Identifying that "he" refers to a previously mentioned person.
- **Noun Phrase Coreference:** Linking "the president" to a specific person's name.
- **Demonstratives:** Resolving "this" or "that" to a previous concept.

Resources: Porter Stemmer

The **Porter Stemmer** is a widely used, rule-based algorithm in Natural Language Processing (NLP) for reducing English words to their base or root form (stem) by removing common suffixes. It is known for its simplicity, speed, and effectiveness in applications like information retrieval and text mining.

Key Characteristics and Functionality

- **Rule-Based:** The algorithm applies a series of fixed rules in five sequential steps to strip suffixes like "-ing," "-ed," "-es," and "-ly".
- **Mathematical Approach:** It classifies characters as consonants or vowels and uses a "measure" (the number of vowel-consonant groups) to decide if a suffix should be removed, ensuring a systematic process.
- **Efficiency:** Due to its simple, rule-based nature, it is fast and efficient for processing large amounts of text data.
- **English-Specific:** It is designed specifically for the English language and does not work well with other languages.
- **Inexact Stems:** The output stem is not always a real, meaningful dictionary word (e.g., "happily" becomes "happi"; "universal" becomes "univers").

Implementation in Python (NLTK)

The Porter Stemmer is readily available in the popular Python Natural Language Toolkit (NLTK) library.

Python

```
import nltk
from nltk.stem import PorterStemmer

# Initialize the Porter Stemmer
porter_stemmer = PorterStemmer()

# List of words to stem
words = ["running", "eats", "eaten", "history", "finally"]

# Apply the stemmer
stemmed_words = [porter_stemmer.stem(word) for word in words]

# Print output
print(stemmed_words)
# Output: ['run', 'eat', 'eaten', 'histori', 'final']
```

Advantages and Limitations

Advantages:

- **Reduces Vocabulary Size:** Consolidates different word forms ("running", "runs") into a single stem ("run"), which reduces the number of unique words in a dataset.
- **Improves Information Retrieval:** Enhances search engine functionality by matching variations of a query word to relevant documents.
- **Simple and Fast:** Easy to implement and processes text quickly, making it suitable for large-scale projects.

Limitations

- **Over-stemming/Under-stemming:** Can sometimes be too aggressive and cut off too much of a word ("university" → "univers"), or not enough, leading to potential loss of meaning or inconsistencies.
- **Inaccurate Stems:** The resulting stem is not always a valid English word.
- **No Context:** It operates without considering the word's context, unlike lemmatization

Related Resources and Alternative Algorithms

For different use cases, other stemming algorithms or lemmatization may be more appropriate.

- **Snowball Stemmer (Porter2):** An improved, faster version of the Porter stemmer that supports multiple languages.
- **Lancaster Stemmer:** A more aggressive stemmer that often produces shorter stems but can be less accurate.
- **Lemmatization:** A more sophisticated technique that uses vocabulary and morphological analysis to return the base or dictionary form (lemma) of a word, which is always a valid word (e.g., "am," "are," "is" all become "be"). This approach is generally more accurate but slower than stemming.

Lemmatizer

A lemmatizer in NLP is a text pre-processing tool that reduces inflected words to their dictionary base form, known as a lemma (e.g., "better" becomes "good", "running" to "run"). Unlike stemming, it uses vocabulary and morphological analysis to determine the intended context and part-of-speech, improving accuracy for tasks like sentiment analysis and information retrieval, though it is computationally slower.

Key Aspects of Lemmatizers

- **Definition & Purpose:** Lemmatization ensures that a word is reduced to its actual, valid dictionary word, whereas stemming may produce non-words by simply cutting off suffixes. It is critical for improving the accuracy of NLP models by grouping different forms of the same word.
- **Process:** The process considers the context, part-of-speech (POS) tags, and morphological analysis, allowing "better" to become "good" or "saw" to be mapped to "see" or "saw" depending on whether it is a noun or verb.

Popular Libraries & Tools:

- **NLTK (WordNetLemmatizer):** A commonly used Python library for Lemmatization that provides a simple interface to normalize text.
- **spaCy:** Known for fast, efficient lemmatization and built-in POS tagging.
- **TextBlob & StanfordNLP:** Additional libraries for advanced text processing.
- **Applications:** Used extensively in chatbots, text mining, and information retrieval systems to map different word forms to a common topic.
- **Disadvantages:** Lemmatization is more computationally intensive and slower than stemming, which may make it less suitable for some real-time applications.

Comparison: Stemming vs. Lemmatization

- **Stemming:** Removes suffixes ("studies" → "studi").
Lemmatization: Uses dictionaries and morphological analysis ("studies" → "study").

Penn Treebank

The Penn Treebank (PTB) is a landmark corpus of over 4.5 million words of American English, annotated with part-of-speech (POS) tags, phrase structures (syntactic trees), and semantic labels. It is a foundational dataset for training and evaluating statistical NLP models, particularly for syntactic parsing and tokenization.

Key Aspects of the Penn Treebank

- **Significance:** It established standard methodologies for NLP, including large-scale annotation, inter-annotator agreement metrics, and strict training/testing splits.
- **Annotations:**
 - **POS Tagging:** Uses a commonly used 36-tag set, often referred to as the Penn Treebank POS tagset.
 - **Syntactic Structure:** Provides phrase-structure trees using bracketed labels (e.g., S for sentence, NP for noun phrase, VP for verb phrase).
 - **Predicate-Argument Structure:** Incorporates "empty categories" (nodes that don't dominate words) to represent trace elements, such as in passive voice or skipped words.

Usage in NLP:

- **Parsing:** Used to train parsers to identify the hierarchical structure of sentences.
- **Tokenization:** Defines methods for splitting text, including handling clitics (e.g., "I'm" "I" + "'m") and punctuation.
- **Speech Processing:** Includes over 1.6 million words of transcribed, annotated spoken text.

Brill's Tagger

Brill's Tagger is a transformative, rule-based Part-of-Speech (POS) tagging algorithm introduced by Eric Brill in 1992 that assigns grammatical categories (nouns, verbs, etc.) by initially applying simple frequency-based rules and then iteratively correcting errors based on learned contextual rules. It achieves 95-97% accuracy, combining the efficiency of rule-based systems with the learning capabilities of statistical methods.

Key Aspects of Brill's Tagger:

- **Transformation-Based Learning (TBL):** The tagger operates by learning a set of ordered rules that transform an initial, often incorrect, tagging into a more accurate one.
- **Process:**
 1. **Initial Tagging:** It assigns the most frequent tag to each word (or a default 'noun' for unknown words).
 2. **Error-Driven Rules:** The system compares the current tags with a "gold standard" (tagged corpus) and learns rules that fix the most errors while creating the fewest new ones.
 3. **Iteration:** Rules are applied sequentially to the text to refine the tagging.
- **Rule Example:** A rule might state: "Change tag from NN (noun) to VB (verb) if the previous word is 'to'".
- **Advantages:** It is highly interpretable (human-readable rules), efficient, and does not require large probability tables like hidden Markov models.

- **Limitations:** It can be susceptible to overfitting the training set and requires a labeled corpus to learn the transformation rules.

It is frequently used in NLTK (Natural Language Toolkit) for English, and is considered effective for languages with similar structures to English.

WordNet

WordNet is a large, lexical database of English, acting as a dictionary designed for Natural Language Processing (NLP) that groups words into sets of synonyms called synsets. It organizes nouns, verbs, adjectives, and adverbs into a network of semantic relationships (hypernyms, antonyms, meronyms) to help computers understand word meaning, context, and relationships.

Key Aspects of WordNet in NLP:

- **Synsets (Cognitive Synonyms):** Words are grouped by their underlying concepts. For example, {car, automobile, motorcar} form a single synset representing one concept.
- **Semantic Relationships:** It maps connections between words, such as "is-a" (hypernym/hyponym - *Dog* is a *canine*) and "part-of" (meronym/holonym - *beak* is part of a *bird*)
- **Components:** Includes definitions (glosses) and example sentences for many synsets.
- **Usage in NLP Tasks:**
 - **Word Sense Disambiguation:** Determining which meaning of a word is used in a sentence.
 - **Semantic Similarity:** Calculating how closely related two words or concepts are.
 - **Information Retrieval:** Expanding search queries using synonyms to improve results.

WordNet is commonly accessed via the `nltk.corpus.wordnet` module in Python.

PropBank

PropBank (Proposition Bank) is a critical NLP resource that annotates the Penn Treebank with semantic roles, mapping "who did what to whom, when, where, and why". By focusing on verb-argument structures (predicates) rather than just syntax, it enables robust Semantic Role Labeling (SRL), improving machine understanding of sentence meaning.

Key Aspects of PropBank

- **Semantic Role Labeling (SRL):** PropBank provides the training data necessary for systems to automatically identify the semantic arguments (e.g., Agent, Patient) associated with a verb, which is a key component of semantic analysis.
- **Annotation Structure:** It adds a layer of annotation to the syntactic structure of the Penn Treebank, defining core arguments (Arg0-Arg5) and adjuncts (ArgM-LOC, ArgM-TMP, etc.).
- **Verb-Centric:** PropBank focuses on the argument structure of verbs, offering a, domain-independent resource for training, which is crucial for robust natural language understanding.
- **Universal PropBanks:** The project has expanded beyond English to include PropBank annotations for 23 languages, often using automatic projection from English.
- **Relation to Other Resources:** While PropBank Wikipedia is centered on specific verb instances, it differs from FrameNet Wikipedia, which uses more abstract, frame-based definitions.

Applications and Impact

- **Semantic Role Labeling (SRL):** PropBank is the foundational resource for training and evaluating SRL systems.
- **Information Extraction:** It enables machines to extract structured information from unstructured text.
- **Machine Translation:** PropBank annotations are used to improve accuracy in translation tasks.

FrameNet

FrameNet is a computational linguistics resource based on frame semantics, providing a structured database of over 1,200 semantic frames and 13,630+ lexical units to represent word meanings and participant roles (frame elements) in context. It is used in NLP to understand "who did what to whom, when, where, and why" for tasks like information extraction, question answering, and machine translation.

Key aspects of FrameNet in NLP:

- **Frame Semantics:** Meaning is understood through conceptual structures (frames) representing events, relations, or states.
- **Components:** Contains **Lexical Units (LUs)** (words), **Frame Elements (FEs)** (roles/participants), and **Frame-to-Frame Relations**.
- **Applications:** Used for semantic role labeling (SRL), sentiment analysis, and enhancing machine understanding of text.
- **Language Support:** While English-centric, it is used for cross-lingual applications and has been extended to languages like Spanish and Portuguese.
- **Resources:** It offers manually annotated sentences (over 22,225) to train machine learning models.

Brown Corpus

The Brown Corpus, created in 1961 at Brown University, is the first million-word electronic corpus of American English, featuring 500 samples across 15 genres. It is a foundational, POS-tagged dataset used in NLP for training models, parsing, and linguistic research, often accessed via the NLTK library.

Key Aspects of the Brown Corpus:

- **Structure:** Contains $\approx$ 1 million words from 500 written English sources published in 1961.
- **Genres:** Covers diverse categories including news, editorial, fiction, academic, and government documents.
- **Usage in NLP:** Primarily used for training, testing, Part-of-Speech (POS) tagging, and language modeling.
- **Legacy:** Set the standard for later corpora, such as the LOB Corpus (British English).
- **Accessibility:** Available through the NLTK library for Python, allowing for tasks like frequency analysis and POS tagging.

Although considered small by modern standards, the Brown Corpus remains a key, structured dataset for analyzing standard American English.

British National Corpus (BNC)

The BNC is distributed in a format which makes possible almost any kind of computer-based research on the nature of the language. Obvious application areas include lexicography, natural language understanding (NLP) systems, and all branches of applied and theoretical linguistics.

The British National Corpus (BNC) is a large digital database of English language that contains about 100 million words collected from many different sources.

These texts come from:

- Books
- Newspapers
- Academic articles
- Conversations
- Speeches
- Magazines

Most of the language in the BNC represents British English used in the late 20th century.

Purpose of BNC in NLP

The BNC helps researchers and students to:

- Study how English words are used in real life
- Analyze grammar and sentence structures
- Build language models
- Train NLP systems such as speech recognition, machine translation, and chatbots

Example

If we want to study how the word “bank” is used in different contexts, the BNC provides many real sentences where the word appears. This helps computers understand different meanings of the same word.

British National Corpus (BNC):

A large collection of about 100 million words of written and spoken British English used for research and development in Natural Language Processing and linguistics.